RAJALAKSHMI ENGINEERING COLLEGE

V Y Aares 251801001-2025-AIDS V2

Started on	Thursday, 27 August 2026, 8:52 AM
State	Finished
Completed on	Thursday, 27 August 2026, 8:52 AM
Time taken	22 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Ascending order comparator
int compareAsc(const void *a, const void *b) {
    long long int_a = *(const long long *)a;
    long long int_b = *(const long long *)b;
    if (int_a < int_b) return -1;
    if (int_a > int_b) return 1;
    return 0;
}

// Descending order comparator
int compareDesc(const void *a, const void *b) {
    long long int_a = *(const long long *)a;
    long long int_b = *(const long long *)b;
    if (int_a > int_b) return -1;
    if (int_a < int_b) return 1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long *arr1 = (long long *)malloc(n * sizeof(long long));
    long long *arr2 = (long long *)malloc(n * sizeof(long long));

    // Read elements for array_One
    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr1[i]);
    }

    // Read elements for array_Two
    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr2[i]);
    }

    // 1. Sort array_One ascending
    qsort(arr1, n, sizeof(long long), compareAsc);

    // 2. Sort array_Two descending
    qsort(arr2, n, sizeof(long long), compareDesc);

    // 3. Compute minimum product sum
    long long minSum = 0;
    for (int i = 0; i < n; i++) {
        minSum += arr1[i] * arr2[i];
    }

    printf("%lld\n", minSum);

```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course